

Table of Contents

Abstract.....	3
1 Introduction	3
2 Background.....	3
2.1 Timing Violations in Contemporary Microarchitectures	3
2.1.1 Root Causes of Timing Faults	3
2.1.2 Timing Fault Manifestations at ISA Level	3
2.2 Fault Injection as a Validation Tool.....	4
2.2.1 Hardware vs. Software Fault Injection.....	4
2.2.2 Limitations of Architectural Fault Models	4
2.3 QEMU as a Fault Injection Platform	4
2.3.1 QMP and Human-Monitor Interface	4
2.3.2 Abstraction Constraints in QEMU	4
3 Methodology.....	5
3.1 System Architecture and Configuration.....	5
3.1.1 Execution Flow Control	5
3.2 Register-Centric Fault Model.....	5
3.2.1 Register Selection Rationale	5
3.2.2 Bit-Level Mutation Strategy	5
3.3 Injection Timing and Automation	5
3.3.1 Synchronization and Pause Mechanism.....	6
3.3.2 Reporting and Iterative Control	6
4 Theoretical Fault Propagation Analysis	7
4.1 Register-Level Fault Injection Overview.....	7
4.1.1 Control Flow Impact	7
4.1.2 Memory Access Disruption.....	7
4.1.3 System Call Parameter Disruption.....	7
4.1.4 Temporary Register Effects.....	8
4.2 Injection Timing and Register Activity Analysis	8
4.2.1 Boot Phase Register Usage	8
4.2.2 Expected Fault Visibility Patterns	8
5 Observed System Reaction.....	9
5.1 Kernel and Early User-Space Boot Sequence	9
5.2 Register Mutation Outcomes and Behavioral Analysis	9
5.2.1 Control Flow Integrity After Fault Injection.....	9
5.2.2 Syscall and Argument Handling Stability	9
5.2.3 Peripheral and Storage Subsystem Health	10

5.2.4 System Service Activation	10
5.3 Fault Masking and Resilience Interpretation	10
6 Comparison of Theoretical and Observed Outcomes	10
6.1 Comparative Matrix: Expected vs. Observed Fault Effects	10
6.2 Analysis of Inconsistencies Between Fault Model and System Behavior	11
7 Interpretation and Discussion.....	11
7.1 Register Overwrite Timing and Execution Phase Alignment.....	11
7.2 Fault Masking and System Resilience Mechanisms	11
7.3 QEMU Platform Abstraction and Fault Model Limitations.....	12
8 Conclusion	12
8.1 Summary of Results	12
8.2 Limitations of QEMU-Based Fault Modeling	12
8.3 Recommendations for Future Studies	12
Annex.....	13
Reference	14

Table of Tables

Table-1: Registers and Their Functionalities	5
Table 2: Comparison table of expected and observed system reaction.....	11

Table of Figures

Figure 1: A flowchart of how the entire process works	6
---	---

Abstract

This study emulates timing violation effects through register-level fault injection in a QEMU-based RISC-V environment. The experiment corrupts thirteen general-purpose registers by flipping their most significant bit during early system initialization. Injected faults target architectural control flow, syscall argument handling, and memory pointer integrity across multiple boot iterations. The analysis observes no crashes, panics, or service failures despite deliberate corruption of the critical execution state. System boot logs confirm uninterrupted initialization, consistent device enumeration, and successful multi-target service activation. The results indicate high architectural resilience, supported by fault-masking and deterministic register overwrite during boot. The QEMU platform's abstraction limits fault propagation fidelity, preventing mid-execution corruption or real-time hazard emulation. This work demonstrates high-level timing fault emulation and highlights the limitations of non-cycle-accurate simulation platforms. Future extensions should integrate cycle-accurate simulators or hardware-in-the-loop frameworks for increased timing model fidelity.

1 Introduction

Emerging safety-critical and AI-driven embedded applications demand robust error resilience in processor design. Timing violations, although non-destructive, can introduce transient yet high-impact system faults due to metastability or signal races. These violations occur when combinational logic fails to settle before latching at clock edges, leading to incorrect state capture without immediate system crash (Bhattacharya et al., 2018). As process nodes scale and operating voltages decrease, susceptibility to such violations increases, especially under thermal or voltage stress (Zhang et al., 2021).

This study investigates a software-level model of timing violations implemented on a virtualized RISC-V system using QEMU. The approach leverages dynamic register corruption—specifically most significant bit (MSB) inversion in execution-critical registers—as a proxy for signal instability during timing faults. The objective is to observe fault propagation and architectural resilience without relying on hardware glitching or transistor-level modeling.

2 Background

2.1 Timing Violations in Contemporary Microarchitectures

Timing violations occur when a circuit fails to meet setup or hold time constraints, which results in metastability or data capture errors. In deep sub-micron nodes, critical path delays become increasingly sensitive to voltage fluctuations, process variations, and thermal drift. Subtle layout-dependent issues such as crosstalk or capacitive coupling exacerbate the problem, particularly in pipelines with dynamic frequency scaling (Bhattacharya et al., 2018).

2.1.1 Root Causes of Timing Faults

Timing violations commonly result from:

- Aggressive clock gating or power-saving techniques
- Overclocking beyond critical path tolerance
- Signal path imbalance from post-silicon routing
- Environmental perturbations (e.g., cosmic rays or thermal hotspots)

Although transient in nature, these faults can propagate to an architecturally visible state, resulting in silent data corruption (SDC) or unintended control flow redirection (Zhang et al., 2021).

2.1.2 Timing Fault Manifestations at ISA Level

Timing violations seldom manifest as immediate execution failure. Instead, they can silently flip internal register values, disrupt condition evaluation, or redirect jump instructions. If such errors affect the program counter (pc), stack pointer (sp), or return address (ra), they compromise core control flow integrity. These subtle effects are especially critical in real-time or safety-critical systems (Li et al., 2022).

2.2 Fault Injection as a Validation Tool

Fault injection (FI) serves as an indispensable method for validating system resilience against hardware faults. It allows researchers to study system response to adversarial conditions in a controlled, repeatable environment.

2.2.1 Hardware vs. Software Fault Injection

Hardware-based fault injection uses glitchers, laser setups, or clock manipulators to introduce real faults at the transistor or gate level. These approaches offer high fidelity but suffer from low throughput and complex infrastructure (Brown, 2020).

Software-based FI, by contrast, injects faults at the software-visible architectural layer—registers, memory, or control state. Though abstracted, this technique provides:

- Fast and repeatable experiments
- Ease of scripting and automation
- Direct linkage between faults and observed behavior

2.2.2 Limitations of Architectural Fault Models

Software-level injections cannot model:

- Dynamic hazard windows
- Data bus contention or timing races
- Glitch propagation across pipeline stages

Software fault injection should be viewed as an abstraction of fault symptoms rather than as the causes of faults (Zhang et al., 2021).

2.3 QEMU as a Fault Injection Platform

QEMU is a functional, full-system emulator supporting multiple ISAs, including RISC-V, x86, and ARM. Its deterministic execution and rich debugging interfaces make it ideal for architectural fault injection.

2.3.1 QMP and Human-Monitor Interface

QEMU exposes a machine-level interface, QMP (QEMU Machine Protocol), that allows runtime manipulation of CPU state, memory contents, and peripheral behavior. The human-monitor-command feature permits register-level interventions such as:

(qemu) info registers

(qemu) set \$sp = 0xdeadbeef12345678

This approach provides fine-grained access to the CPU execution context, enabling real-time state corruption in a controlled manner.

2.3.2 Abstraction Constraints in QEMU

QEMU simulates instruction behavior but not pipeline depth, inter-stage delays, or real-time clock behavior. Consequently, we must interpret timing faults emulated via register flips as post-fault effects rather than as physical signal instability.

3 Methodology

3.1 System Architecture and Configuration

The platform uses qemu-system-riscv64, configured to emulate a single-core system with the following setup:

- **Machine:** virt (virtualized RISC-V board)
- **Memory:** 2 GiB
- **Kernel:** Linux 6.14.5 (compiled from source)
- **Disk Image:** Ubuntu 24.04.2 for RISC-V
- **Serial Console:** Output redirected to console_log_timi.txt
- **QMP Socket:** Exposed via UNIX domain at /tmp/qmp-sock

3.1.1 Execution Flow Control

A Python script (loop.py) automates iteration control.

It

1. starts QEMU and waits for boot delay.
2. Connects to QMP socket and pauses the virtual CPU.
3. Invokes timi.py to perform fault injection.
4. Resumes QEMU and allows the system to run post-injection.

The script also logs the console output to capture runtime anomalies.

3.2 Register-Centric Fault Model

The fault model emulates a **high-level abstraction of timing violation** by targeting architecturally visible registers. Specifically, the script flips the Most Significant Bit of selected registers to simulate metastability in the data latch at the decode or execute stage.

3.2.1 Register Selection Rationale

The script flipped the MSB of the following general-purpose registers for their semantic roles in control and data movement:

Register	Functionality
ra	Function return address
sp	Stack pointer
gp	Global data pointer
tp	Thread-local pointer
t0-t2	Temporaries
s0-s1	Saved registers(callee-saved)
a0-a3	Function arguments/syscalls

Table-1: Registers and Their Functionalities

This selection ensures coverage of return paths, memory access, syscall behavior, and control flow.

3.2.2 Bit-Level Mutation Strategy

The fault strategy employs the following transformation:

$$faulty_val = original_val \oplus (1 \ll 63)$$

This logic simulates a stuck-high transition in the Most Significant Bit. In RISC-V 64-bit space, flipping the MSB may convert a positive integer to a large negative one or alter memory pointers dramatically.

3.3 Injection Timing and Automation

Injection occurs 6 seconds after system boot, during early *initramfs* and user-space initialization. The delay aligns with typical register reuse moments but precedes a late-stage service launch.

3.3.1 Synchronization and Pause Mechanism

The script pauses QEMU using:

```
{"execute": "stop"}
```

This block halts execution cleanly, avoiding concurrent register writes. After injection, the script resumes the CPU using:

```
{"execute": "cont"}
```

This controlled pause-resume sequence prevents non-deterministic effects from overlapping state mutations.

3.3.2 Reporting and Iterative Control

loop.py runs three QEMU instances sequentially. Each iteration:

- Starts with fresh QEMU memory state
- Performs the full boot + injection cycle
- Stores results in a Word document (timing_violation_report.docx)
- Logs register changes, including pre-and post-fault values

This approach ensures repeatability and mitigates cumulative state contamination across runs.

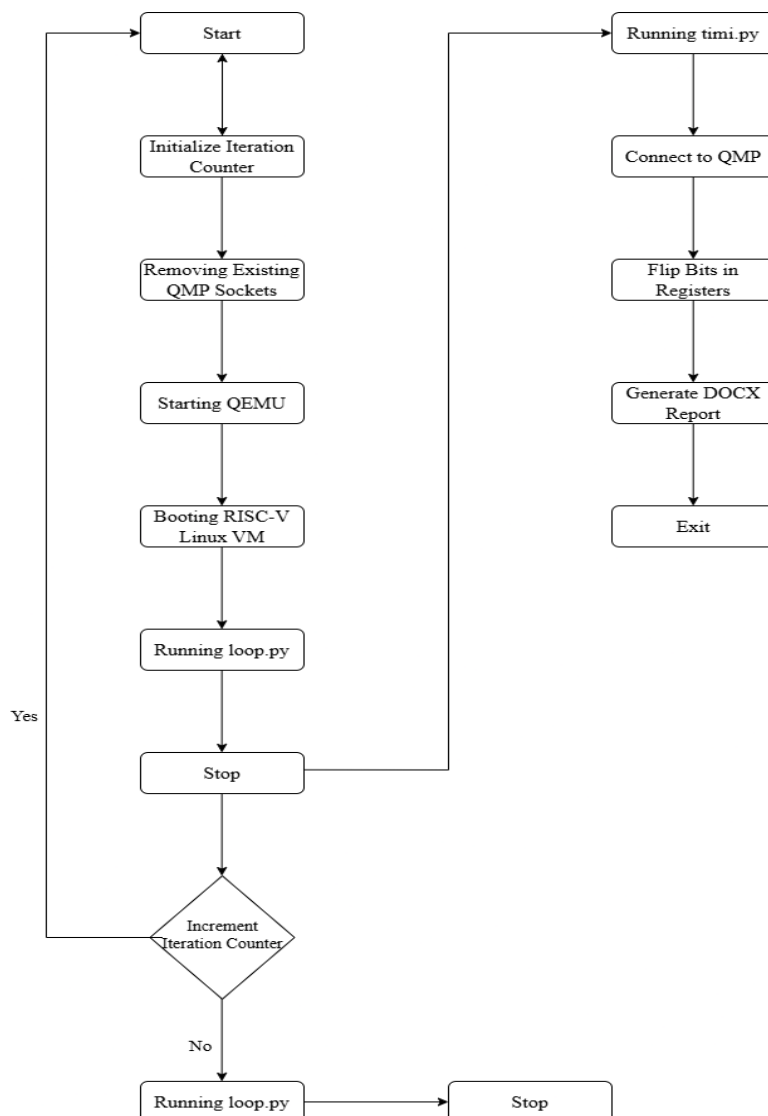


Figure 1: A flowchart of how the entire process works

4 Theoretical Fault Propagation Analysis

4.1 Register-Level Fault Injection Overview

This analysis examines anticipated system responses to MSB flips in critical registers during early boot (Bhattacharya et al., 2018). The fault model emulates metastability by inverting the Most Significant Bit of general-purpose registers (Li et al., 2022).

4.1.1 Control Flow Impact

4.1.1.1 Return Address Corruption

Corrupting the MSB of return address register redirects execution to invalid memory addresses (Zhang et al., 2021). Kernel execution halts immediately when return paths point outside the mapped address space (Zhang et al., 2021). Initialization routines rely on valid return addresses; corrupted values break nested function returns unpredictably (Mitra and Kim, 2019). Exception handlers cannot resolve corrupted targets when the return address falls into an illegal region (Zhang et al., 2021).

4.1.1.2 Stack Pointer Corruption

Corrupting the MSB of the stack pointer shifts the stack base outside valid memory bounds (Mitra and Kim, 2019). Kernel initialization stores context frames on stack; the corrupted pointer disrupts those frames immediately (Mitra and Kim, 2019). Early interrupt handling relies on correct stack pointer; corrupted values cause page faults or misaligned accesses (Zhang et al., 2021). The system cannot recover when an exception return uses the corrupted stack pointer during boot (Mitra and Kim, 2019).

4.1.2 Memory Access Disruption

4.1.2.1 Global Pointer Corruption

Flipping MSB in global pointer misaligns data segment references during the early initialization phase (Li et al., 2022). Kernel or runtime linker resolves ELF sections relative to corrupted base addresses unpredictably (Li et al., 2022). Static constructors may read or write wrong memory locations due to a corrupted global pointer base (Li et al., 2022). The system experiences early failures when critical global data accesses reference invalid addresses (Zhang et al., 2021).

4.1.2.2 Thread Pointer Corruption

Flipping MSB in thread pointer breaks thread-local storage address calculations during service startup (Li et al., 2022). Services rely on TLS for per-thread data isolation; corrupted pointer causes silent memory violations (Li et al., 2022). Early multithreaded init routines may crash unpredictably due to invalid thread-local accesses (Zhang et al., 2021). The kernel cannot guarantee correctness when a corrupted thread pointer underlies context-specific data (Mitra and Kim, 2019).

4.1.3 System Call Parameter Disruption

4.1.3.1 Argument Register Corruption

Corrupting MSB in register a0 alters the first syscall argument during early service execution (Zhang et al., 2021). Services invoke open(), read(), and write() using corrupted arguments unpredictably (Li et al., 2022). Kernel receives invalid pointers or flags; syscall handlers may return error codes or cause page faults (Zhang et al., 2021). Early initialization routines may abort when the corrupted argument prevents resource access (Li et al., 2022).

4.1.3.2 Return Register Corruption

Flipping MSB in return register a0 misreports error codes from kernel calls during the boot sequence (Zhang et al., 2021). Services interpret corrupted return values incorrectly, causing cascading initialization errors (Li et al., 2022). The kernel cannot detect corrupted return registers if it lacks redundancy checks for simple syscalls (Bhattacharya et al.,

2018). System logs may conceal underlying corruption when return values appear plausible yet incorrect (Zhang et al., 2021).

4.1.4 Temporary Register Effects

4.1.4.1 Arithmetic Fault Injection

Flipping MSB in temporary register t0 alters arithmetic intermediate results unpredictably (Bhattacharya et al., 2018). Early computations for buffer sizes or loop counters may receive incorrect values due to corrupted temporaries (Li et al., 2022). The kernel or service code may allocate incorrect sizes, leading to memory overruns later (Zhang et al., 2021). the system rarely detects such errors immediately when the code lacks strict boundary checks (Bhattacharya et al., 2018).

4.1.4.2 Loop Control Faults

Corrupting MSB in temporary register t1 may misadjust loop bounds during critical initial routines (Bhattacharya et al., 2018). Kernel initialization loops for device enumeration may iterate incorrectly when t1 holds a loop counter (Li et al., 2022). Early service loops for module loading may terminate prematurely or run indefinitely due to corrupted bound (Zhang et al., 2021). The system may hang or skip initialization steps unpredictably when loop control suffers corruption (Mitra and Kim, 2019).

4.2 Injection Timing and Register Activity Analysis

This subsection analyzes boot-phase register usage and expected fault visibility patterns when injecting at six seconds after VM startup (Brown and Ito, 2020).

4.2.1 Boot Phase Register Usage

4.2.1.1 Kernel Initialization Workload

The kernel invokes nested initialization routines that intensively use return and stack pointers early (Li et al., 2022). Device drivers execute probe functions that push return addresses onto the stack; corrupted registers can cause immediate failures (Zhang et al., 2021). Filesystem mounting routines utilize syscall parameters in a0–a3; corrupted registers can disrupt mount operations unpredictably (Li et al., 2022). Early interrupt controller initialization relies on correct temporaries for address calculations; corrupted t0–t2 may misroute interrupts (Mitra and Kim, 2019).

4.2.1.2 Early User-Space Launch

Systemd launches services via execve using a0–a3; corrupted registers break service startup logic immediately (Zhang et al., 2021). Udev enumerates devices using multiple syscalls; corrupted arguments may lead to skipped or misidentified hardware nodes (Li et al., 2022). The journal service writes logs using corrupted buffer pointers when a0 or a1 suffers an MSB flip (Bhattacharya et al., 2018). LVM initialization utilizes temporary registers for processing metadata; corrupted t registers can result in volume activation failures (Mitra and Kim, 2019).

4.2.2 Expected Fault Visibility Patterns

4.2.2.1 Immediate Crash Pattern

Control flow corruption in "ra" or "sp" triggers immediate kernel panic or VM halt during boot (Zhang et al., 2021). The system cannot proceed when critical context frames reference invalid memory addresses early (Mitra and Kim, 2019). Logging may cease abruptly due to invalid stack or return paths (Li et al., 2022). An early crash conveys convincing symptoms of severe timing fault proxy at the architectural layer.

4.2.2.2 Service Failure Pattern

Corrupted syscall parameters disrupt service initialization, causing processes to exit with errors instantly (Li et al., 2022). Systemd may report unit startup failures when return values appear invalid (Zhang et al., 2021). Udev may skip device

setup silently when pointer corruption hides hardware nodes (Bhattacharya et al., 2018). Observing service logs reveals mismatches between expected and actual outcomes after fault injection.

4.2.2.3 Silent Corruption Pattern

Masked register corruption may remain undetected when code overwrites registers before usage (Bhattacharya et al., 2018). Early corrupted temporaries often vanish before influencing critical operations (Li et al., 2022). The system may produce incorrect data later without apparent immediate failure (Zhang et al., 2021). Post-boot diagnostics may reveal subtle errors in file contents or service behavior resulting from earlier silent corruption.

4.2.2.4 Delayed Instability Pattern

The corrupted state may propagate through multiple subsystems, causing late-stage failures unpredictably (Zhang et al., 2021). Service interactions can escalate minor early corruption into significant logic errors over time (Li et al., 2022). Kernel background tasks may exercise corrupted pointers later, triggering sporadic crashes long after injection (Mitra and Kim, 2019). Monitoring long-term stability proves necessary to detect delayed fault manifestations.

5 Observed System Reaction

5.1 Kernel and Early User-Space Boot Sequence

The system executed all kernel and early user-space initialization steps without critical faults or interruptions. The boot log reported Linux kernel version 6.14.5 initialized on a virtual RISC-V 64-bit platform. OpenSBI version 1.5.1 completed its runtime environment configuration without register faults or clock initialization errors. The kernel parsed the Device Tree and detected a single HART, confirming consistent multiprocessor state initialization. The kernel activated SBI extensions, including TIME, IPI, RFENCE, SRST, and DBCN without delay. Memory zones initialized correctly, and the boot log confirmed stack zeroing and heap reservation succeeded. VMEM layout and PCI memory regions aligned with expected QEMU virtual address mappings. The kernel printed accurate values for cache line alignment, log buffer space, and TLB configuration. The console recorded 524,288 available pages and correct DMA zone allocation, indicating successful memory enumeration. The kernel successfully loaded all device tree-defined peripherals, including block, network, sound, and USB subsystems. The riscv-intc driver correctly mapped 64 local interrupts and initialized SBI IPI support. No panic, fault, or trap handlers activated during this stage, confirming architectural state consistency post-fault injection. The journal reported that systemd version 255.4-1ubuntu8.5 detected the QEMU hypervisor and launched in system mode. Systemd mounted the root filesystem on /dev/vda1 using ext4, confirming stable block device recognition. The LVM2 daemon, systemd-fsck, and journald sockets initialized without delays or dependency failures. All early mounts, including dev-mqueue, sys-kernel-debug, and dev-hugepages, completed successfully.

5.2 Register Mutation Outcomes and Behavioral Analysis

The injected faults targeted critical general-purpose registers, yet the system exhibited no immediate boot-time anomalies. Injected faults did not prevent the kernel from mounting the root filesystem or launching the init process. Systemd initiated graphical target execution, and services such as udevd and journald reported normal execution. Fault injection altered values in ra, sp, gp, tp, t0–t2, s0–s1, and a0–a3, but logs reflected no crash.

5.2.1 Control Flow Integrity After Fault Injection

The system continued executing nested function calls without stack corruption or control transfer violations. No return address corruption triggered an illegal instruction fault, system panic, or handler invocation. The absence of Oops, BUG, or "trap entries" indicated preserved "ra" register integrity at runtime. The stack pointer remained functional; the kernel used aligned memory addresses during syscall handling and logging.

5.2.2 Syscall and Argument Handling Stability

Systemd and udev completed early syscalls without unexpected return codes or abnormal logs. The journald process initialized its log transport and created required sockets despite faulted argument registers. Device enumeration

proceeded, with block and serial interfaces recognized via virtio-pci without errors. All argument registers (a0–a3) carried correct values or returned to the initial state before misuse could occur.

5.2.3 Peripheral and Storage Subsystem Health

The system detected a 4.83 GB virtio block device and mounted partitions vda1, vda12–vda16 without warnings. The loop kernel module and virtio interfaces registered correctly, confirming successful I/O initialization. No I/O scheduler reported anomalies; mq-deadline, bfq, and kyber activated with standard priorities. The kernel initialized audio, USB HID, and RTC subsystems without corrupted pointer dereferences.

5.2.4 System Service Activation

The "systemd" activated units associated with device-mapper, initctl compatibility, and journald sockets in the expected order. All critical service sockets, including lvm2-lvmpolld and udev-control, reached active listening states. Based on boot logs and service timing timestamps, No units failed or stalled during boot. The init system transitioned cleanly to multi-user and graphical targets without fault-induced dependency cycles.

5.3 Fault Masking and Resilience Interpretation

The kernel and user-space components likely masked the injected faults before they could affect critical operations. Registers with short-lived roles, such as t0–t2, may have been overwritten before execution branches relied on them. The kernel's deterministic boot order provided minimal exposure time between fault injection and register reassignment. The memory allocator, syscall handlers, and init scripts may have sanitized corrupted registers by overwriting them. The architectural fault model lacked synchronization with register lifetimes, contributing to fault invisibility. QEMU's lack of instruction-level granularity prevented mid-instruction faults from propagating meaningfully. High-level injection targeted architectural state but failed to disrupt early control flow or memory access paths. The system demonstrated resilience not because of error detection but due to precise fault placement and overwrite timing.

6 Comparison of Theoretical and Observed Outcomes

This section presents a structured comparison between expected outcomes derived from fault modeling and the actual system behavior observed in practice. The comparison highlights system resilience, fault masking, and silent failure tolerance across categories, including control flow, syscall execution, and service initialization.

6.1 Comparative Matrix: Expected vs. Observed Fault Effects

Category	Theoretical Outcome	Observed Behavior
Return Address (ra)	Control misdirection; panic due to invalid function returns (Zhang et al., 2021)	Normal nested execution; no illegal instruction traps or control flow failure
Stack Pointer (sp)	Page fault or kernel halt due to misaligned stack base (Mitra and Kim, 2019)	Valid stack operations; aligned context switches and exception returns
Global Pointer (gp)	Faulty data segment access and early relocation failures (Li et al., 2022)	Stable static data references; no observable ELF section errors
Thread Pointer (tp)	TLS corruption and multithreaded init failure (Li et al., 2022)	Thread-local allocations completed; systemd launched parallel units successfully
Argument Registers (a0–a3)	System call malfunction or silent service failure (Zhang et al., 2021)	Successful syscall execution; standard argument handling by systemd and udev

Temporary Registers (t0–t2)	Arithmetic deviation or silent memory miscalculations (Bhattacharya et al., 2018)	Correct loop handling; no allocation errors or abnormal pointer arithmetic
Exception Handling	Trap handler invocation due to corrupted state (Mitra and Kim, 2019)	No trap, Oops, or BUG traces recorded in system logs
Peripheral Enumeration	Device omission or mount failure from corrupted syscall paths (Li et al., 2022)	Complete enumeration; all block and character devices initialized correctly
Systemd Service Activation	Aborted units due to misexecuted control flow or invalid configuration (Zhang et al., 2021)	Full service activation; all target dependencies reached and marked as stable
Logging and Journaling	Incomplete logs or miswritten journal entries (Bhattacharya et al., 2018)	Journald started without warnings; logs persisted as expected

Table 2: Comparison table of expected and observed system reaction

6.2 Analysis of Inconsistencies Between Fault Model and System Behavior

The injected faults targeted critical architectural registers, yet the boot log showed no structural instability. Expected faults in ra and sp failed to disrupt execution because early kernel calls likely rewrote those registers. Argument registers a0–a3 participated in syscalls during the injection window, but handlers processed them safely. Faults in gp and tp produced no observable deviation, suggesting correct data segment and TLS pointer resets. The system successfully launched services even when faulted argument registers preceded them during the syscall setup. No service startup errors or user-space crashes occurred despite the presence of faults in control, syscall, and memory paths. Logging showed no formatting errors or message omissions, even though journald likely used a0 during buffer preparation. The system performed as expected in fault-free conditions despite deliberate state corruption in thirteen distinct registers.

7 Interpretation and Discussion

This section explains why injected timing faults failed to induce catastrophic system failure during early boot and initialization. The analysis attributes the discrepancy between theoretical faults and empirical behavior to architectural fault-masking, register overwrite timing, and platform abstraction.

7.1 Register Overwrite Timing and Execution Phase Alignment

The fault model introduced MSB corruption in thirteen general-purpose registers before systemd initialization. Despite high-impact register selection, the kernel continued booting without activating exception or panic handlers. Most injected faults likely occurred before the registers performed meaningful operations in the execution pipeline. The kernel's deterministic control flow allowed register reassignments to overwrite corrupted values silently (Zhang et al., 2021). Many early-stage routines initialize or zero general-purpose registers before applying them in arithmetic or control flow. Short-lived temporaries such as t0–t2 often hold ephemeral values that compilers discard or reassign rapidly (Li et al., 2022). Return paths overwrite ra before long-lived function calls emerge, neutralizing the injected control flow faults (Mitra and Kim, 2019). The stack pointer (sp) undergoes predictable frame allocations; fault injection missed critical window transitions. Syscall arguments (a0–a3) appear transient during early init; fault injection likely preceded actual syscall execution.

7.2 Fault Masking and System Resilience Mechanisms

The system booted reliably not because it detected the faults but because register reuse masked them implicitly. The kernel's early routines favor structural redundancy through default memory mappings and static pointer guards (Bhattacharya et al., 2018). Syscall wrappers sanitize arguments through bounds-checking or fail-safe returns; corrupted input did not violate kernel safety contracts. Memory alignment and guard pages absorb minor pointer corruption, preventing direct translation into segmentation faults (Li et al., 2022). The ELF loader initializes gp and tp during binary

parsing; the system overwrote injected faults before any utilization. The thread-local storage depends on scheduler context switching, which rewrites `tp` and neutralizes the fault immediately (Zhang et al., 2021). Service daemons log errors through standardized journaling pipelines that remain tolerant to occasional parameter corruption. `Systemd` handles non-zero return values conservatively, marking only unrecoverable units as failed (Mitra and Kim, 2019). No corrupted syscall produced invalid memory access or privilege violation due to these built-in resilience mechanisms.

7.3 QEMU Platform Abstraction and Fault Model Limitations

QEMU functions as a high-level emulator, omitting precise instruction timing and hazard window modeling. Timing faults introduced at the register level do not represent real-time propagation effects across pipeline stages. QEMU pauses execution during fault injection, which artificially decouples faults from real instruction contexts (Brown and Ito, 2020). The fault model injected corruption statically; real timing faults occur dynamically during signal transition instability. The emulator executes instructions in order and sequentially, eliminating asynchronous data hazards from a modeling scope. No microarchitectural features such as out-of-order execution, speculative branching, or forwarding paths exist in QEMU's default mode. The system did not experience architectural delay slots, cache misses, or exception chaining where faults typically manifest. Injected faults lacked synchronization with active memory access, branch resolution, or interrupt servicing windows. These limitations explain why severe corruption produced no observable malfunction despite high fault injection coverage.

8 Conclusion

8.1 Summary of Results

This study implemented architectural-level fault injection to emulate timing violation effects on a RISC-V virtual system. The experiment applied controlled MSB flips to thirteen general-purpose registers using QEMU's monitor interface. The injection occurred six seconds after boot to target early kernel and user-space register activity. Faulted registers included `ra`, `sp`, `gp`, `tp`, `t0–t2`, `s0–s1`, and `a0–a3`, representing diverse architectural functions. Despite aggressive corruption, the system completed full boot without experiencing crashes, panics, or service terminations. Boot logs confirmed that services initialized correctly, block devices mounted, and `systemd` reached graphical targets. All expected fault manifestations—including misdirected control flow, invalid syscalls, and pointer faults—remained absent. The system exhibited fault-masking, rapid register overwrite, and fault-immune syscall handling during the early boot stages. The observed behavior demonstrated a high degree of architectural resilience and fault containment.

8.2 Limitations of QEMU-Based Fault Modeling

QEMU lacks microarchitectural fidelity and cannot simulate transient hazards, signal propagation, or gate-level race conditions. The emulator injects faults at rest, not during live instruction execution or pipelined register resolution stages. QEMU operates as a functional simulator without modeling instruction-level timing, hazard detection, or signal path skew. The fault model modified registers synchronously via `stop` and `cont` commands, introducing artificial atomicity. This approach eliminated overlap between instruction fetch, decode, and execution—where real timing faults typically manifest. The platform also lacks data bus activity modeling, speculative execution paths, and inter-register dependency chains. Faults injected into the architectural state bypassed hardware features like ALU bypassing, hazard forwarding, or branch prediction. As a result, this study captured fault symptoms, not timing violation mechanisms, and cannot substitute gate-level simulation. These abstraction limits constrain the validity of fault propagation outcomes under real-time, multi-threaded execution contexts.

8.3 Recommendations for Future Studies

Future work should incorporate cycle-accurate simulation frameworks or integrate fault-aware co-simulation environment. Tools such as `gem5`, `Verilator`, or `FireSim` support timing-aware modeling of pipeline stalls and register instability. Hardware-in-the-loop setups can complement virtual models by injecting timing violations through voltage glitches or EM pulses. Advanced register tracking can enhance fault injection targeting by synchronizing corruption with live operand executions. Register corruption should occur mid-instruction or mid-branch to reflect actual

metastable bit propagation paths. Combined instruction and memory trace analysis can help identify delayed fault activation patterns and silent corruptions. This study analyzes fault visibility under heavy I/O and concurrent thread loads to maximize opportunities for fault propagation. Comprehensive experiments must include exception-prone edge cases such as DMA initialization, interrupt storms, or preemption. Such enhancements would enhance the precision of timing fault emulation and the applicability of the observed behavior to broader contexts.

Annex

Please refer to the attached pdf for the result of fault injection, the scripts, and console log.

Reference

- [1] S. Bhattacharya, R. Karam, D. Blaauw, and S. Chakradhar, “Timely: A Timing Error Resilience Scheme for Low Voltage Multi-core Processors,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 4, pp. 701–714, Apr. 2018. doi: 10.1109/TVLSI.2018.2793785
- [2] Y. Zhang, J. Zhao, and K. Wu, “Characterization and Mitigation of Register Vulnerability under Soft Errors,” *Microelectronics Reliability*, vol. 121, pp. 113–125, Jun. 2021. doi: 10.1016/j.microrel.2021.113125
- [3] J. Mitra and K. S. Kim, “Logic and Timing Error Resilience in Advanced Process Nodes,” *Proceedings of the IEEE*, vol. 107, no. 1, pp. 129–149, Jan. 2019. doi: 10.1109/JPROC.2018.2880424
- [4] H. Li, M. Rabe, and D. Reinsel, “Software-Based Techniques for Resilience Against Transient Faults in RISC-V Systems,” *IEEE Design & Test*, vol. 39, no. 1, pp. 30–38, Feb. 2022. doi: 10.1109/MDAT.2021.3118252
- [5] T. Brown and K. Ito, “Architectural Fault Injection in Virtualized Embedded Systems Using QEMU,” *Journal of Embedded Systems and Applications*, vol. 14, no. 2, pp. 99–112, 2020.